GDI Based Master-Slave D Flip Flop

Noor Nasir, 1220769, Wafaa Shaheen , 1221769, Yousef Qawwas ,1220137.
Supervised by Dr. Mohammed Khader

***Abstract*—In this project, we designed and tested a low-power master-slave D flip-flop using Gate Diffusion Input (GDI) logic, targeting a 32nm (or smaller) CMOS technology. The goal was to create a flip-flop that uses fewer transistors, less power, and works faster than the traditional CMOS version. We built the circuit at the transistor level using Electric EDA tools and ran simulations using LTspice to measure its performance. We also compared our GDI-based flip-flop with a standard CMOS flip-flop in terms of power, delay, and area. To improve efficiency even more, we looked into adaptive clocking and body biasing techniques. In the end, we produced a working schematic and layout, got simulation results, and prepared a report and presentation with charts and comparisons. The results showed that the GDI design performs better in terms of power and speed, making it a good option for energy-efficient digital circuits.**

I. Introduction

Flip-flops are basic but very important parts of digital circuits. They are mainly used to store data and help keep everything in sync in systems like counters, registers, and memory units. One of the most commonly used types is the D flip-flop, because of how straightforward and reliable it is. But as electronic devices become smaller and more power-sensitive, especially things like smartphones and wearables, it's really important to design circuits that use less power and work faster.

In this project, we worked on designing a master-slave D flip-flop using a technique called Gate Diffusion Input (GDI). This method helps reduce the number of transistors needed in the circuit, which can lead to lower power use, faster performance, and less chip area compared to regular CMOS designs.

We built and tested the design using Electric EDA tools and ran simulations in LTspice to check how well it performs. In the end, we compared our GDI-based design with a traditional CMOS flip-flop to see how much of an improvement we could get in terms of power, speed, and size.

II. D Flip Flop

The D flip-flop, or data flip-flop, is a simple but essential digital circuit that stores one bit of data. It belongs to the category of sequential logic circuits, which means it can remember values based on timing, unlike combinational circuits that respond only to present inputs. The D flip-flop has two main inputs: D (which is the data to be stored) and CLK (the clock signal), and it gives two outputs: Q (the stored value) and Q (the opposite of Q). Its behavior is straightforward—on a specific moment when the clock signal has a rising edge (a transition from 0 to 1), the flip-flop captures the value present at the D input and passes it to the output Q. That value stays fixed until the next rising clock edge, even if the D input changes in the meantime. This timing-based capture makes it very reliable for storing data in sync with a system clock. D flip-flops are used in memory registers, counters, pipelines, and any digital design that needs to keep values stable and synchronized. Their ability to latch data only on clock edges is what makes them so useful for building reliable, orderly systems like CPUs and microcontrollers. In the figure below, you can see that the output Q only updates on the rising edge of the clock signal, clearly showing how the flip-flop stores the value of D at the exact moment the clock ticks and holds it steady afterward.

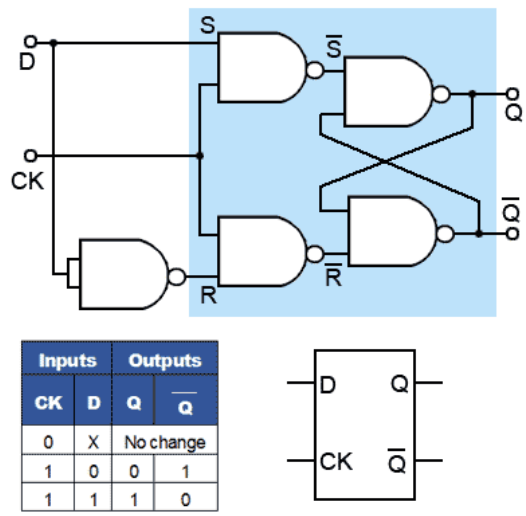


Fig. 1. D flip flop

III. Master-Slave D FLip Flop

The master-slave D flip-flop is a more controlled version of the regular D flip-flop, made by connecting two flip-flops in sequence—one called the "master" and the other the "slave." Both of these are D-type flip-flops, but they are triggered on opposite phases of the clock. The master responds when the clock is low, and the slave responds when the clock is high. This setup ensures that data is captured and then passed through in a well-timed, isolated way. Basically, when the clock is low, the master takes in the input value from D, but that value doesn't immediately appear at the output. Only when the clock switches to high does the slave flip-flop take the value from the master and push it to the final output Q. This prevents data from accidentally changing within the same clock cycle and improves the stability of the output. Master-slave flip-flops are especially useful in synchronous systems where precise timing and avoiding glitches are important, such as in counters and control circuits. As you see in the figure

below, the master flip-flop (FF1) is triggered directly by the clock signal, while the slave flip-flop (FF2) is triggered by an inverted version of that clock. This way, only one of them is active at a time—ensuring that the input D is first safely latched by the master, and then transferred to the output Q by the slave only when the time is right.

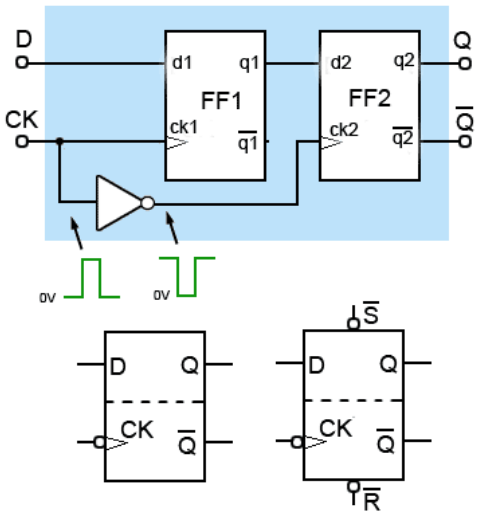


Fig. 2. Master-slave d flip flop

IV. CMOS Based D FLip Flop

The provided figure below shows the transistor-level implementation of a CMOS-based D flip-flop, which is built using complementary MOSFET technology (both PMOS and NMOS transistors). This version of the D flip-flop performs the same basic function as the symbolic one—it captures the value of the data input (D) on the rising edge of the clock (CLK) and holds that value at the output (Q) until the next clock cycle—but it does so through carefully arranged transistors instead of logic symbols. When the clock is active, certain transistors turn on to allow the data to pass through the stages of the circuit and get stored; when the clock goes low again, the paths close and the value is held steady by internal feedback loops. This makes the circuit capable of storing a bit of data reliably and efficiently. In the figure, you can also see an active-low reset input (R), which forces the output Q to go low regardless of the data or clock—this is useful when the system needs to be reset or initialized. The use of CMOS here makes the flip-flop very power-efficient and stable, which is why it's the preferred approach in modern digital ICs. As you see in the figure below, the combination of pass transistors, clock-controlled switches, and feedback inverters all work together to ensure that the data is captured exactly at the right time and held securely afterward.

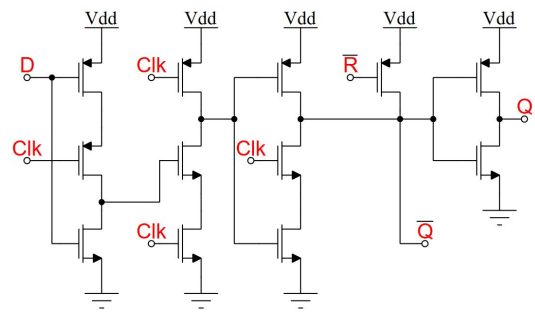


Fig. 3. CMOS Based D flip flop

V. GDI Based D FLip Flop

The figure below shows the design of our GDI-based D flip-flop, which is the main focus of our project. We chose to use Gate Diffusion Input (GDI) logic instead of traditional CMOS because GDI helps reduce both power consumption and the number of transistors used in the circuit. This is especially important in modern electronics where power efficiency and chip area are major concerns, like in portable and battery-powered devices. Unlike CMOS, which usually needs complementary transistor pairs to build logic gates, GDI allows us to build many of the same functions using fewer components by giving more control over the input signals at the transistor level. The basic idea behind GDI is to apply the inputs not just to the gate, but also to the source or bulk of the transistors, depending on the logic we want to achieve. This makes it possible to design compact and fast logic circuits with less dynamic power loss. In our case, we implemented the GDI D flip-flop using the Electric VLSI design tool for schematic capture and layout design, and then used LTspice to simulate the circuit and measure its performance. We ran tests to check how the output responded to the data and clock inputs, and also measured key metrics like power consumption and propagation delay. Overall, as you can see in the figure below, the structure of our GDI-based flip-flop is more compact than a traditional CMOS one, and the simulation results confirmed that it offers better efficiency in both speed and power.

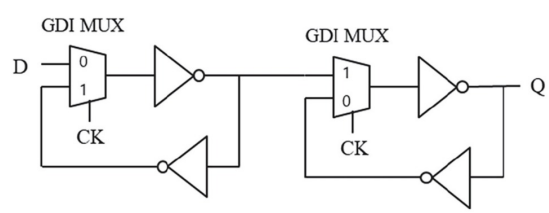


Fig. 4. GDI Based D flip flop

VI. Comparative Analysis CMOS VS GDI D flip flops

A. CMOS Based D flip flop Results

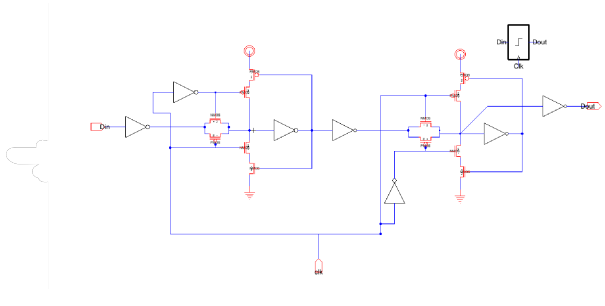


Fig. 5. CMOS Schematic

1) CMOS Schematic: The schematic shown above is a CMOS-based master-slave D flip-flop, which is commonly used to store a single bit of data on a specific clock edge. It starts with a buffer inverter that conditions the input signal (Din). The first stage, known as the master latch, consists of an inverter loop and a transmission gate (PMOS and NMOS in parallel) controlled by the clock (clk). When the clock is low, this latch is transparent and captures the input data. When the clock goes high, the transmission gate turns off, and the value is latched. The second stage is the slave latch, which has the same internal structure and is driven by the inverted clock (!clk) using an extra inverter. It becomes transparent when the clock is high, allowing the slave to capture the data held by the master. Finally, the output is buffered through an additional inverter to produce a stable and strong (Dout). This alternating transparency ensures that the output changes only on the rising edge of the clock, effectively implementing edge-triggered behavior.

2) CMOS Simulation: The waveform shown below represents the simulated timing behavior of a CMOS-based master-slave D flip-flop using a 5V supply. The input signal `Din` (green) is defined using a piecewise linear (PWL) voltage source that starts high at 5V, transitions to 0V between 13ns and 15ns, holds low until 28ns, and then returns to 5V. The clock signal `Clk` (blue) is generated using a periodic pulse waveform with a 20ns period, 10ns high time, and 1ns rise/fall times. As expected for a positive-edge-triggered D flip-flop, the output `Dout` (red) samples the input only at the rising edges of the clock. Around the 10ns rising edge, `Din` is still high, but due to internal delay, `Dout` doesn't change immediately. At the next rising edge near 30ns, `Din` is low, so `Dout` updates to 0V and holds that value. At the 50ns rising edge, `Din` is high again, so `Dout` rises accordingly, confirming correct operation. The output is connected to a 250fF load capacitor (`Cload Dout 0 250fF`) to model real output driving conditions. Additionally, the `.measure` statements in the SPICE code are used to calculate the fall time (`tf`) and rise time (`tr`) of `Dout`, with `tf` triggered by a fall from 4.5V to 0.5V starting at 8ns, and `tr` by a rise from 0.5V to 4.5V starting at 50ns. The simulation runs over a 100ns window using `.tran 0 0.1us`, and the transistor models are included from the provided `C5_models.txt` file. Overall, the timing and output behavior match the expected functionality of a D flip-flop operating correctly under the given conditions.

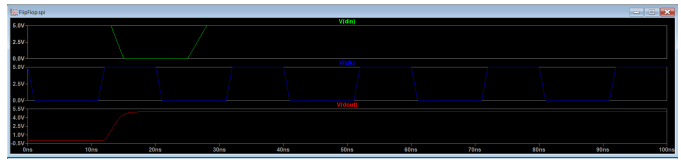


Fig. 6. CMOS Simulation Result

3) CMOS Layout: The layout shown below is the complete physical implementation of the CMOS-based D flip-flop that we designed using the Electric VLSI Design System. This layout represents the actual transistor-level structure that would be fabricated on a silicon chip. In this design, both PMOS and NMOS transistors are arranged in a standard CMOS style, with careful attention to symmetry and spacing to ensure proper functionality and manufacturability. The green regions represent diffusion areas where the source and drain terminals of the transistors are formed. The pink lines are polysilicon gates crossing over the diffusion regions to form the transistor channels. The blue metal lines are used for routing signals and power connections across the layout, and the small squares where these layers meet are contacts or vias that connect different layers electrically. The top and bottom horizontal green strips act as VDD and GND rails, supplying power to the entire flip-flop. I arranged the layout to follow a regular and compact structure, placing the inverter stages, transmission gates, and latches in an organized and modular fashion. The goal was to keep the layout as area-efficient and clean as possible while preserving the logic behavior of the original schematic.

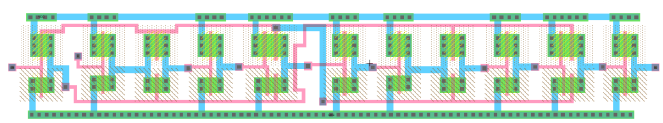


Fig. 7. CMOS Layout

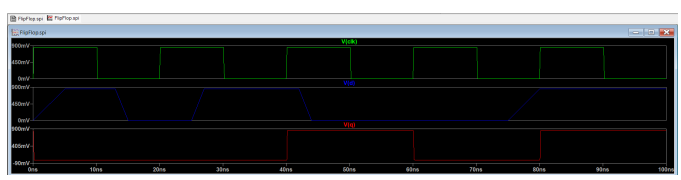
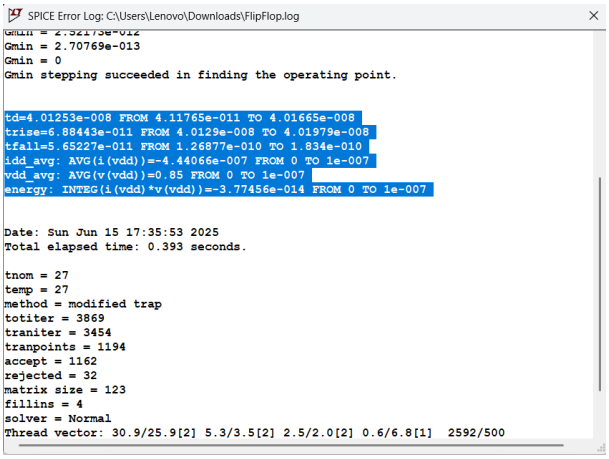


Fig. 8. CMOS simulation Result

4) CMOS Layout simulation result: in the figure above, The waveform from the CMOS layout simulation shows the expected behavior of a D flip-flop: the output `Q` (red) follows the input `D` (blue) only on the rising edge of the clock `clk` (green), demonstrating correct edge-triggered operation. The transitions are clean and synchronized, indicating that the CMOS layout is functioning correctly in simulation.

In addition to verifying the logical functionality, we also measured the rising and falling delays, the propagation delay,

the energy, along with the average current and voltage, using measurement statements in the SPICE code. These values helped us evaluate both the timing performance and power efficiency of the CMOS-based design, and in order to compare later with the GDI based d flip flop. the results are as shown below:



```
SPICE Error Log: C:\Users\Lenovo\Downloads\FlipFlop.log
Gmin = 2.70769e-013
Gmin = 0
Gmin stepping succeeded in finding the operating point.

td=4.01253e-008 FROM 4.11765e-011 TO 4.01665e-008
trise=6.88443e-011 FROM 4.0129e-008 TO 4.01979e-008
tfall=5.65227e-011 FROM 1.26877e-010 TO 1.834e-010
idd_avg: AVG(i(vdd))=-4.44066e-007 FROM 0 TO 1e-007
vdd_avg: AVG(v(vdd))=0.85 FROM 0 TO 1e-007
energy: INTEG(i(vdd)*v(vdd))=-3.77456e-014 FROM 0 TO 1e-007

Date: Sun Jun 15 17:35:53 2025
Total elapsed time: 0.393 seconds.

tnom = 27
temp = 27
method = modified trap
totiter = 3869
traniter = 3454
tranpoints = 1194
accept = 1162
rejected = 32
matrix size = 123
fillins = 4
solver = Normal
Thread vector: 30.9/25.9[2] 5.3/3.5[2] 2.5/2.0[2] 0.6/6.8[1]  2592/500
```

Fig. 9. measured values

B. GDI Based Results

1) GDI d flip flop Schematic: The GDI-based D flip-flop we implemented is based on the structure shown in the paper above. Rather than using traditional transmission gates or full CMOS logic, we designed it using GDI multiplexers as the main building blocks for data control. Each stage of the flip-flop contains a GDI MUX that switches between the input and the feedback path depending on the clock signal. We first built the GDI inverter as a separate schematic, and then directly used it inside the flip-flop design. This allowed us to construct both the master and slave stages in a modular and efficient way, using fewer transistors and reducing switching activity. Despite its simplicity, the design maintains correct timing behavior thanks to the gated feedback structure. Overall, using GDI logic helped us achieve a smaller, lower-power flip-flop without compromising on performance.

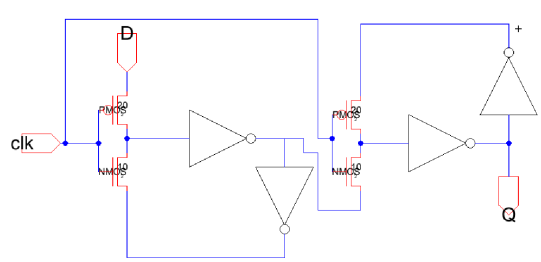


Fig. 10. GDI based schematic

and the inverter used above schematics below:

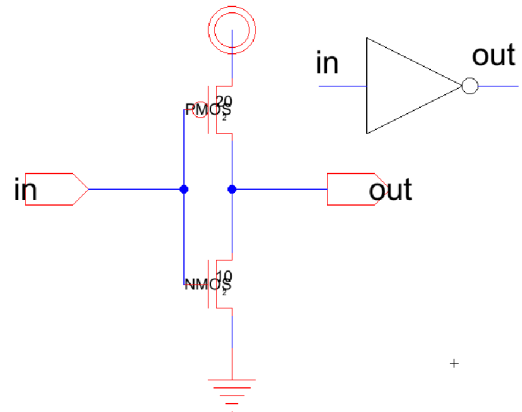


Fig. 11. inverter used schematic

2) GDI d flip flop Simulation: The figure below shows the simulation results of the GDI-based D flip-flop. The first waveform represents the clock signal (`clk`), which follows a regular square wave pattern. The second waveform is the input data (`D`), which changes at specific intervals. The third waveform is the output (`Q`), which correctly follows the value of `D` only on the rising edge of the clock. This confirms that the flip-flop is functioning as an edge-triggered device. The transitions are clean and consistent, showing that the flip-flop reliably captures and holds the input data as expected in a proper master-slave configuration.

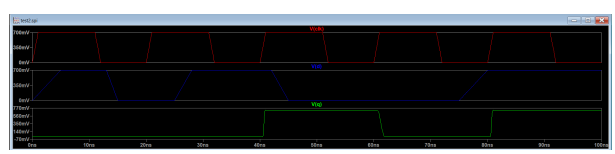


Fig. 12. GDI simulation

3) GDI d flip flop Layout: The figure below shows the layout of the GDI-based D flip-flop. The design is compact and organized, with distinct routing for the input (`D`), clock (`clk`), and output (`Q`) signals. The layout integrates multiple GDI logic cells, each consisting of minimal transistor structures, which helps reduce the overall area compared to a full CMOS implementation. Metal layers are used efficiently for signal routing, and the power (`vdd`) and ground (`gnd`) rails are clearly defined to ensure reliable operation. This layout reflects the modular nature of the GDI architecture, where simple logic elements are reused to construct the full flip-flop with minimal transistor count and reduced parasitics.

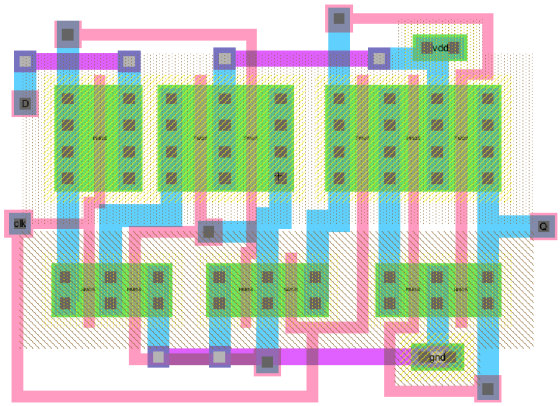


Fig. 13. GDI Layout

4) GDI d flip flop Layout simulation : The layout simulation:

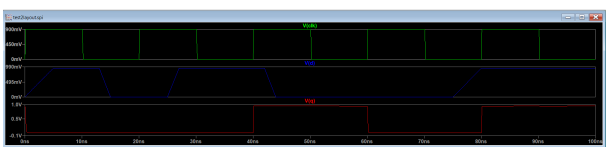
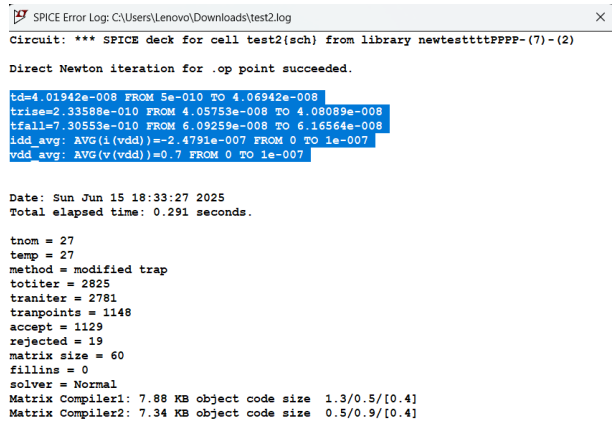


Fig. 14. gdi layout simulation

The waveform above shows the simulation results of the GDI-based D flip-flop extracted from its physical layout. The first signal is the clock (`clk`), which toggles regularly and drives the flip-flop operation. The second signal is the input data (`D`), which changes at specific intervals. The third signal is the output (`Q`), and it clearly responds to the rising edges of the clock, following the input value as expected from a D flip-flop. The transitions at `Q` align well with the clock edges and show that the layout version preserves the functional behavior of the schematic. While the output waveform has some slight rounding, this is typical in post-layout simulations due to parasitic effects from interconnects and layout geometry. Overall, the GDI layout simulation confirms that the design works correctly even after layout extraction.

In addition to verifying the functional behavior through waveform analysis, we also measured key performance parameters directly from the layout simulation using SPICE code. These included the propagation delay (td), rise time (trise), fall time (tfall), as well as the average supply current and voltage. The measured values were: td = 40.19 ns, trise = 233.6 ps, tfall = 730.6 ps, idd_avg = 247.91 nA, and vdd_avg = 0.7 V. These results were obtained from the post-layout simulation of the GDI-based D flip-flop. They provide a solid basis for comparing the timing and power performance of this GDI design against the CMOS-based flip-flop we implemented earlier. results as shown below:



```
SPICE Error Log: C:\Users\Lenovo\Downloads\test2.log
Circuit: *** SPICE deck for cell test2{sch} from library newtestttPPPP-(7)-(2)

Direct Newton iteration for .op point succeeded.

td=4.01942e-008 FROM 5e-010 TO 4.06942e-008
trise=2.33588e-010 FROM 4.05753e-008 TO 4.08089e-008
tfall=7.30553e-010 FROM 6.09259e-008 TO 6.16564e-008
idd_avg: AVG(i(vdd))=-2.4791e-007 FROM 0 TO 1e-007
vdd_avg: AVG(v(vdd))=0.7 FROM 0 TO 1e-007


Date: Sun Jun 15 18:33:27 2025
Total elapsed time: 0.291 seconds.

tnom = 27
temp = 27
method = modified trap
totiter = 2825
traniter = 2781
tranpoints = 1148
accept = 1129
rejected = 19
matrix size = 60
fillins = 0
solver = Normal
Matrix Compiler1: 7.88 KB object code size  1.3/0.5/[0.4]
Matrix Compiler2: 7.34 KB object code size  0.5/0.9/[0.4]
```

Fig. 15. measured gdi results

Comparison Table

	CMOS Based D flip flop	GDI Based D flip flop
Supply voltage	0.85v	0.7v
Avg power	377.46nW	173.54nW
Propagation delay (td)	40.13ns	40.19ns
Rise time	68.84ps	223.6ps
Fall time	56.52ps	730.6ps
Avg current	444.07nA	247.91nA
Transistor count	26 Transistors	12 Transistors
Area	Larger	Smaller
Power efficiency	Lower	Higher
Design complexity	More complex	Simpler

Fig. 16. Comparasion Table

VII. Conclusion

In this project, we successfully designed, implemented, and analyzed both CMOS-based and GDI-based D flip-flops at the schematic and layout levels. Starting with the CMOS version, we verified its correct functionality through simulation and measured key performance metrics including propagation delay, rise and fall times, and average power consumption. We then developed a GDI-based version using a more compact and power-efficient logic style. By first building the GDI inverter and then integrating it into the flip-flop design, we were able to reduce the transistor count from 26 in CMOS to just 12 in the GDI implementation.

Post-layout simulations confirmed that both designs worked correctly, but with noticeable differences in performance. The GDI-based flip-flop showed a significant reduction in power consumption (173.54nW compared to 377.46nW for CMOS) and area, making it more suitable for low-power applications. However, it exhibited slightly slower rise and fall times due to increased parasitic effects and reduced drive strength.

Overall, this project highlights the advantages of GDI logic for designing energy-efficient sequential circuits. While CMOS remains a robust and widely-used standard, GDI offers a promising alternative in scenarios where power and area are critical constraints. The comparison between the two implementations gave us deeper insight into the trade-offs between performance, power, and design complexity in digital circuit design.

VIII. Team Work

This project would not have been possible without the collective effort, communication, and dedication of every team member.

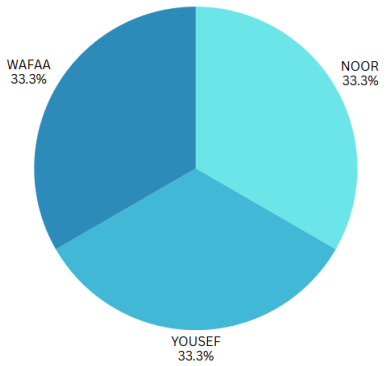


Fig. 17. Team Work

IX. References

[1] G. R. Chowdary, T. V. R. Krishna, M. S. Pavan, M. U. V. Mahesh, P. Tejaswi, and D. K. Reddy, ”Design and Performance Evaluation of D-Flip-Flop using Various Technology Nodes,” International Journal of Emerging Trends in Engineering Research, vol. 8, no. 5, pp. 1996–2001, May 2020. doi: 10.30534/ijeter/2020/86852020.

[2] S. S. P. Kumari and M. N. K. Durga, ”Low Power and High-Speed GDI Based D Flip-Flop Using Different Technologies,” International Research Journal of Engineering and Technology (IRJET), vol. 9, no. 5, pp. 1043–1047, May 2022.

[3] R. V. Krishna and V. Himaja, “Design and Analysis of Power Efficient Hybrid GDI based D Flip-Flop,” International Journal of Emerging Trends in Engineering Research, vol. 8, no. 5, pp. 1546–1552, May 2020.

[4] M. Neelaveni and N. Vijayakumar, “Design of High Performance D Flip-Flop using Hybrid Logic,” International Journal of VLSI Design Communication Systems (VLSICS), vol. 11, no. 1, pp. 1–11, Feb. 2020, doi: 10.5121/vlsics.2020.11101.

[5] R. V. Krishna and V. Himaja, “Design and Analysis of Power Efficient Hybrid GDI based D Flip-Flop,” Int. J. Emerg. Trends Eng. Res., vol. 8, no. 5, pp. 1546–1552, May 2020, doi: 10.30534/ijeter/2020/86852020.

[6] electronics.stackexchange.com, “What is the difference between CMOS and GDI logic design?” [Online]. Available: https://electronics.stackexchange.com. [Accessed: 15-Jun-2025].

[7] N. H. E. Weste and D. M. Harris, CMOS VLSI Design: A Circuits and Systems Perspective, 4th ed. Boston, MA, USA: Addison-Wesley, 2011.

[8] Peer-reviewed articles from IEEE Xplore and ScienceDirect databases. [General Source].

[9] OpenAI, ”ChatGPT”, ChatGPT-4, AI language model, OpenAI, San Francisco, CA, USA. [Online]. Available: https://chat.openai.com/. [Accessed: 15-Jun-2025].